

# GEN AI MINI PROJECT

## Multi-Step Prompt Workflow

|                           |                    |
|---------------------------|--------------------|
| <b>Name</b>               | SAMPATHI YASWANTH  |
| <b>Student ID</b>         | 99230041039        |
| <b>Programme</b>          | Gen AI             |
| <b>Module</b>             | Prompt Engineering |
| <b>Date of Submission</b> | 05/09/2026         |

---

### Overview

For this task I didn't want to just reuse the review-triage example we went through in class, so I picked a different situation to build the same pipeline around: a customer support ticket that needs to be turned into an urgency report. The idea is the same as what was taught – three prompts, each doing one job, chained together – just applied to a different kind of input. It uses few-shot to extract, chain-of-thought to reason, and JSON mode to format the final result.

## Step 1: Extracting the details (few-shot)

For this step I just need the model to pull out fixed fields from the ticket, so few-shot made the most sense – I gave it two worked examples and let it copy the pattern instead of trying to reason anything out.

Extract the following fields from a support ticket as a bullet list:  
order\_id, promised\_sla, actual\_delay, contact\_attempts, business\_impact, sentiment.  
If a field isn't mentioned, write "not stated".

### Example 1

Ticket: "My package (Order #1123) was supposed to arrive in 2 days on your express plan, but it has now been 6 days late and I have not heard back from 3 emails I sent. I'm a first-time customer and this really soured my first impression."

Extracted:

- order\_id: #1123
- promised\_sla: 2 days (express)
- actual\_delay: 6 days late
- contact\_attempts: 3 emails sent, no reply
- business\_impact: not stated (personal order, first-time customer)
- sentiment: disappointed, first impression soured

### Example 2

Ticket: "Order #9987 arrived a day late but everything was in perfect condition, and support answered my one question within an hour. Happy overall."

Extracted:

- order\_id: #9987
- promised\_sla: not stated
- actual\_delay: 1 day late
- contact\_attempts: 1 question, answered within an hour
- business\_impact: not stated
- sentiment: satisfied, minor issue only

Now extract from this ticket:

Ticket: "{raw\_ticket}"

Extracted:

## Step 2: Deciding how urgent it is (CoT)

Urgency isn't a single fact you can just pull out, it's a judgement call – you have to weigh the delay against the promised SLA, whether support already replied, whether there's money involved, and whether the customer sounds like they'll actually leave. That's why I used chain-of-thought here, and I made sure the prompt actually pastes in step1's real bullet list rather than assuming what it says.

Given these extracted facts about a support ticket:  
{step1}

Think step by step about how urgent this ticket is, considering:

1. How far the actual delay exceeds the promised SLA
2. How responsive support has already been (contact\_attempts)
3. Whether there's a business/financial impact, not just inconvenience
4. Whether the customer signals they may churn or escalate

Go through each point briefly, then end with exactly one line:

"Verdict: <low/medium/high>"

### Step 3: Formatting the final output (JSON mode)

This is the only step that actually gets read by something other than a human, so it's the one place where I need a strict, parseable structure instead of natural language.

Using the extracted facts and the urgency verdict below, output ONLY a single valid JSON object (no markdown fences, no extra commentary) with exactly these keys:

"entities": a list of {"field": ..., "value": ...} objects taken directly from the extraction below  
"urgency" : one of "low", "medium", "high", from the verdict below  
"summary" : one sentence summarising the ticket in plain English

Extracted facts:

{step1}

Urgency verdict:

{step2}

## Running it end to end

I tested this on the ticket below and kept all three outputs so it's clear step2 is actually reading step1's output and not just guessing.

**Ticket:** "Order #58213 still hasn't shipped after 9 days, even though checkout promised 3-day delivery. I've emailed support twice and got no reply. I run a small Etsy shop and this delay is costing me real sales – I need this fixed today or I'm moving to a different supplier."

*Step 1 output:*

```
- order_id: #58213
- promised_sla: 3-day delivery
- actual_delay: 9 days elapsed, order not yet shipped
- contact_attempts: 2 emails to support, no reply
- business_impact: reseller (runs an Etsy shop), losing active sales
- sentiment: frustrated, urgent, threatening to switch supplier
```

*Step 2 output:*

```
1. Delay vs SLA - 9 days against a 3-day promise is 3x the committed
   window, that's a real SLA breach, not just a slip.
2. Support responsiveness - 2 emails with zero reply means the normal
   channel has already failed the customer once.
3. Business impact - this isn't a personal parcel, it's stock for a
   business that's actively losing sales right now.
4. Churn signal - the customer says outright they'll switch suppliers
   "today" if it's not fixed, so this is time-boxed.
Everything here points the same way, nothing about it feels like a
"medium" to me.
Verdict: high
```

*Step 3 output (final JSON, checked with json.loads and it parses fine):*

```
{
  "entities": [
    {"field": "order_id", "value": "#58213"},
    {"field": "promised_sla", "value": "3-day delivery"},
    {"field": "actual_delay", "value": "9 days elapsed, not yet shipped"},
    {"field": "contact_attempts", "value": "2 emails to support, no reply"},
    {"field": "business_impact",
     "value": "reseller (Etsy shop), losing active sales"},
    {"field": "sentiment",
     "value": "frustrated, urgent, threatening to switch supplier"}
  ],
  "urgency": "high",
  "summary": "A reseller's order is 9 days late against a 3-day SLA with
             no support response, and they're threatening to switch suppliers
             today."
}
```

## Chaining the three calls in code

```
raw_ticket = ("Order #58213 still hasn't shipped after 9 days, even "  
             "though checkout promised 3-day delivery. I've emailed "  
             "support twice and got no reply. I run a small Etsy shop "  
             "and this delay is costing me real sales - I need this "  
             "fixed today or I'm moving to a different supplier.")  
  
extract_prompt = EXTRACTION_FEWSHOT_PROMPT.format(raw_ticket=raw_ticket)  
step1 = prompt_llm(extract_prompt)  
  
reason_prompt = URGENCY_COT_PROMPT.format(step1=step1)  
step2 = prompt_llm(reason_prompt)  
  
format_prompt = JSON_FORMAT_PROMPT.format(step1=step1, step2=step2)  
step3 = prompt_llm(format_prompt)  
  
result = json.loads(step3)  # parses without any manual cleanup  
print(step1)  
print(step2)  
print(result)
```

### A quick note

I picked support tickets instead of reviews mainly so this wouldn't look identical to everyone else's submission, but the underlying pipeline is the same three-step idea – extract with few-shot, reason with CoT, and lock the final structure with JSON mode. Step 2 is built directly on top of step1's real output above, and the JSON at the end is valid and loads straight into Python.